```
        {
            return;
        }
        if(resource.CreatedById == appUser.Id)                    ❺
        {
            context.Succeed(requirement);                         ❻
        }
    }
}
```

❶ Implements the necessary base class, specifying the requirement and resource type

❷ Injects an instance of the UserManager<T> class using DI

❸ As well as the context and requirement, you're provided the resource instance.

❹ If you aren't authenticated, appUser will be null.

❺ Checks whether the current user created the Recipe by checking the CreatedById property

❻ If the user created the document, Succeeds the requirement; otherwise, does nothing

This handler is slightly more complicated than the examples you've seen previously, primarily because you're using an additional service, `UserManager<>`, to load the `ApplicationUser` entity based on `ClaimsPrincipal` from the request.

**NOTE** In practice, the `ClaimsPrincipal` will likely already have the `Id` added as a claim, making the extra step unnecessary in this case. This example shows the general pattern if you need to use dependency-injected services.

The other significant difference is that the `HandleRequirementAsync` method has provided the `Recipe` resource as a method argument. This is the same object you provided when calling `AuthorizeAsync` on `IAuthorizationService`. You can use this resource to verify whether the current user created it. If so, you `Succeed()` the requirement; otherwise, you do nothing.